

EC4219: Software Engineering

Lecture 10 — Symbolic Execution

Sunbeom So
2025 Spring

- Automatic test generation approach based on computational logic
- Very widely used for finding software bugs and vulnerabilities.
 - ▶ In Microsoft, 33% of bugs have been discovered using symbolic execution during the development of Windows 7.
- Key idea: execute programs “symbolically” using “symbolic” inputs
 - ▶ More systematic approach compared to random testing
- Similar to verification, but each has different goals and different design choices.
 - ▶ Verification: prove correctness – by overapproximating behaviors (inferring invariants)
 - ▶ Symbolic execution: find error-triggering inputs (counterexamples) – by underapproximating behaviors (e.g., unrolling loops)
- From another perspective, can be viewed as:
 - ▶ a method for disproving partial correctness (finding counterexamples)
 - ▶ an instance of bounded verification

Symbolic Execution in a Nutshell

- Program inputs are represented by symbols (e.g., α , β).
- Maintain a state $(stmt, \sigma, \pi)$ where
 - ▶ $stmt$: the next statement to evaluate
 - ▶ σ : symbolic store (mapping from variables to symbolic values)
 - ▶ π : path constraints (a formula that expresses a sequence of branches taken to reach $stmt$). At the beginning of the analysis, $\pi = true$.

Symbolic Execution in a Nutshell (Cont'd)

Depending on *stmt*, symbolic execution proceeds as follows.¹

- $(x := e)^l$: updates the symbolic store σ by associating x with a symbolic expression e_s (obtained by symbolically evaluating e).

$$((x := e)^l, \sigma, \pi) \rightsquigarrow (\text{succ}(l), \sigma[x \mapsto e_s], \pi)$$

where $e_s = \text{eval}(e)(\sigma)$.

- s^l : if e then $s_1^{l_1}$ else $s_2^{l_2}$: creates (forks) two new states.

$$(s^l, \sigma, \pi) \rightsquigarrow (s_1^{l_1}, \sigma, \pi \wedge e_s) \text{ and } (s_2^{l_2}, \sigma, \pi \wedge \neg e_s)$$

- $\text{assert}^l(e)$: check the satisfiability of e and create a new state.
 - ▶ If $\pi \wedge \neg e_s$ is satisfiable, report the error.
 - ▶ If $\pi \wedge \neg e_s$ is unsatisfiable, e cannot be violated through the path (branch sequence) taken so far.

$$(\text{assert}^l(e), \sigma, \pi) \rightsquigarrow (\text{succ}(l), \sigma, \pi \wedge e)$$

¹We assume each statement has a unique label l .

Example: Symbolic Execution

Execution Tree

```
int double (int v) {  
    return 2*v;  
}
```

1 $x: \alpha, y: \beta$
 $pc: true$

```
void testme(int x, int y) {
```

1
2 $z := \text{double}(y);$

if ($z == x$) {

3
if ($x > y + 10$) {

4 Crash

} else { 5 }

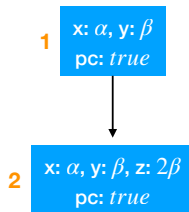
}

6 }

Example: Symbolic Execution (Cont'd)

```
int double (int v) {  
    return 2*v;  
}  
  
void testme(int x, int y) {  
1   z := double (y);  
2  
   if (z==x) {  
3       if (x>y+10) {  
4           Crash  
       } else { 5 }  
   }  
6 }
```

Execution Tree

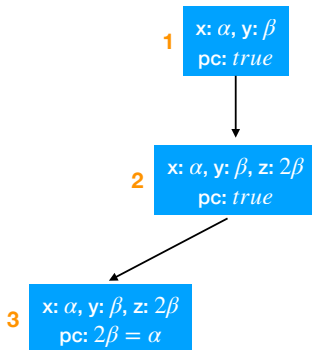


Example: Symbolic Execution (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {  
1   z := double (y);  
2  
   if (z==x) {  
3       if (x>y+10) {  
4           Crash  
       } else { 5  
6       }  
}
```

Execution Tree

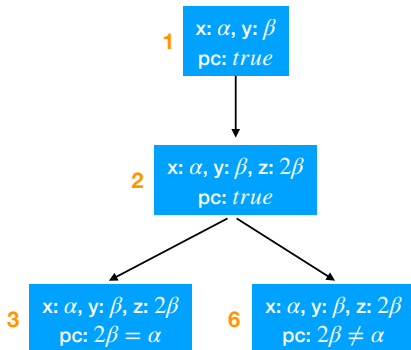


Example: Symbolic Execution (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {  
1   z := double (y);  
2  
   if (z==x) {  
3       if (x>y+10) {  
4           Crash  
       } else { 5 }  
   }  
6 }
```

Execution Tree



Example: Symbolic Execution (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {
```

```
1  z := double (y);  
2
```

```
    if (z==x) {
```

```
3      if (x>y+10) {
```

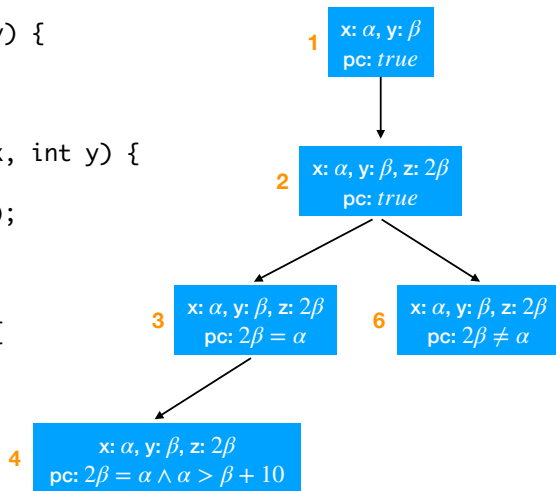
```
4      Crash
```

```
    } else { 5 }
```

```
  }
```

```
6 }
```

Execution Tree



Example: Symbolic Execution (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {
```

```
1   z := double (y);  
2
```

```
   if (z==x) {
```

```
3       if (x>y+10) {
```

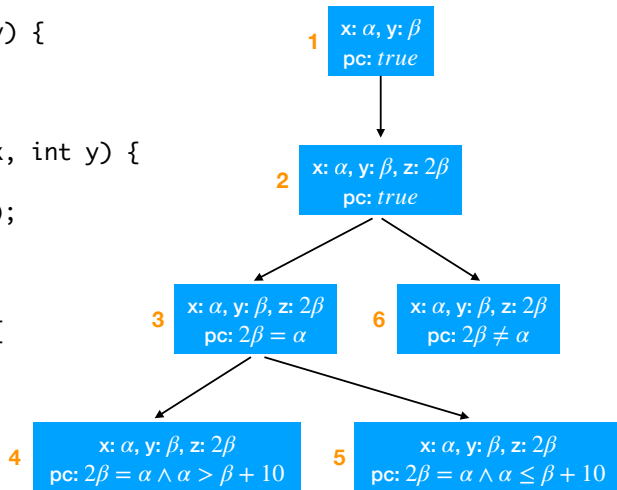
```
4         Crash
```

```
       } else { 5 }
```

```
   }
```

```
6 }
```

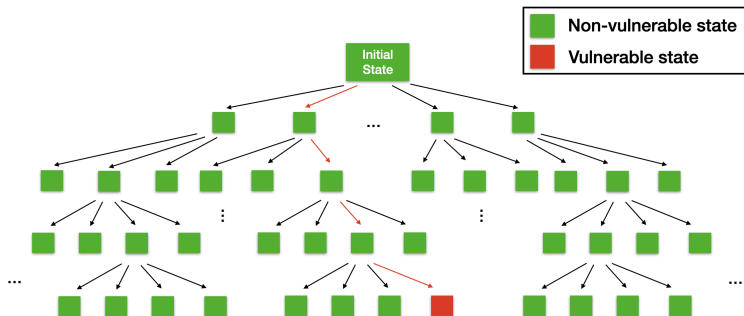
Execution Tree



Challenges in Symbolic Execution (1): Path Explosion

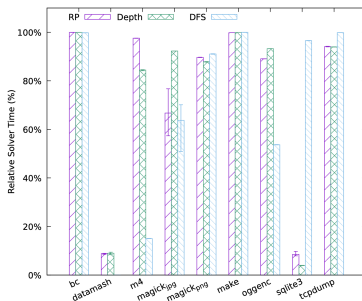
- Given n if/while statements, $\#path = 2^n$ (worst case)
 - E.g., suppose there are multiple invocations of the function f containing 10 if-statements. We already have more than 1M ($= 2^{10} * 2^{10}$) execution paths!

$f(\dots); \dots; f(\dots); \dots$



Challenges in Symbolic Execution (2): Constraint Solving

- Despite the recent advances in SMT solvers, constraint solving is still a major bottleneck.
 - ▶ According to the recent report,² more than 80% of total testing time is spent on SMT solving (invoking a solver and waiting for its response).
- Constraint solving is sometimes practically impossible.
 - ▶ E.g., external function calls whose source code is unknown, hash function calls whose logic is extremely complicated



²Pending Constraints in Symbolic Execution for Better Exploration and Seeding. ASE 2020

Challenges in Symbolic Execution (2): Constraint Solving

Existing SMT solvers are highly unlikely to solve constraints that involve cryptographically secure hash functions (“one-wayness”).

```
int foo (int v) {  
    return secure_hash(v);  
}  
  
void testme(int x, int y) {  
  
    z := foo (y);  
  
    if (z==x) {  
  
        if (x>y+10) {  
            Crash  
        } else {    }  
    }  
}
```

Dynamic Symbolic Execution (Concolic Testing)

- An approach to mitigate the limitations of (pure) symbolic execution (in particular, the difficulty of constraint solving)
- **Concolic**: a compound word of **concrete** and **symbolic**
- Example of hybrid analysis: combines dynamic and static analysis
 - ① Start with random input values
 - ② Concretely execute the program using the inputs. During this step, keep track of both concrete states and symbolic states.
 - ③ Assuming the path constraint $\Phi : \phi_1 \wedge \dots \wedge \phi_n$ upon termination is given, solve the constraint Φ' where one of the conjunct of Φ is negated. E.g., $\Phi' : \phi_1 \wedge \dots \wedge \neg \phi_n$.
 - ④ Concretely execute the program using the inputs obtained by solving Φ' .
 - ⑤ Repeat Step 2–4.
- Symbolic execution (solving Φ') guides concrete execution to increase code coverage.
- Concrete execution guides symbolic execution to simplify constraints (optimize Φ' before solving it).

Example: Concolic Testing

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {
```

```
    ← z := double (y);
```

```
    if (z==x) {
```

```
        if (x>y+10) {
```

Crash

```
        } else { }
```

```
    }
```

```
}
```

Concrete
State

x=22, y=7

Symbolic
State

x= α , y= β
true

1st iteration

Example: Concolic Testing (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {
```

```
    z := double (y);
```

```
    ← if (z==x) {
```

```
        if (x>y+10) {
```

Crash

```
        } else {    }
```

```
    }
```

```
}
```

Concrete
State

x=22, y=7,
z=14

Symbolic
State

x= α , y= β , z=2* β
true

1st iteration

Example: Concolic Testing (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {  
    z := double (y);  
    if (z==x) {  
        if (x>y+10) {  
            Crash  
        } else {  
            }  
        }  
    }  
}
```

Concrete
State

x=22, y=7,
z=14

Symbolic
State

x= α , y= β , z= $2*\beta$
 $2*\beta \neq \alpha$

1st iteration

Example: Concolic Testing (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {  
    z := double (y);  
    if (z==x) {  
        if (x>y+10) {  
            Crash  
        } else {  
        }  
    }  
}
```

Concrete
State

Symbolic
State

- Constraint: $2\beta = \alpha$
- Solution: $\alpha=2, \beta=1$

$x=22, y=7,$
 $z=14$

$x=\alpha, y=\beta, z=2\beta$
 $2\beta \neq \alpha$

1st iteration

Example: Concolic Testing (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {
```

```
    ← z := double (y);
```

```
    if (z==x) {
```

```
        if (x>y+10) {
```

Crash

```
        } else {    }
```

```
    }
```

```
}
```

Concrete
State

x=2, y=1

Symbolic
State

x= α , y= β
true

2nd iteration

Example: Concolic Testing (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {
```

```
    z := double (y);
```

```
    ← if (z==x) {
```

```
        if (x>y+10) {
```

Crash

```
        } else {    }
```

```
    }
```

```
}
```

Concrete
State

x=2, y=1,
z=2

Symbolic
State

x= α , y= β , z=2* β
true

2nd iteration

Example: Concolic Testing (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {
```

```
    z := double (y);
```

```
    if (z==x) {
```

```
        if (x>y+10) {
```

Crash

```
        } else {    }
```

```
    }
```

```
}
```

Concrete
State

x=2, y=1,
z=2

Symbolic
State

x= α , y= β , z= $2*\beta$
 $2*\beta = \alpha$

2nd iteration

Example: Concolic Testing (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {  
    z := double (y);  
    if (z==x) {  
        if (x>y+10) {  
            Crash  
        } else {  
            }  
        }  
    }  
}
```

Concrete
State

$x=2, y=1,$
 $z=2$

Symbolic
State

$x=\alpha, y=\beta, z=2*\beta$
 $2*\beta = \alpha \wedge$
 $\alpha \leq \beta+10$

2nd iteration

Example: Concolic Testing (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {
```

```
    z := double (y);
```

```
    if (z==x) {
```

```
        if (x>y+10) {
```

Crash

```
        } else {    }
```

```
    }
```

```
}
```

Concrete
State

Symbolic
State

- Constraint: $2*\beta = \alpha \wedge \alpha > \beta+10$
- Solution: $\alpha=30, \beta=15$

$x=2, y=1,$
 $z=2$

$x=\alpha, y=\beta, z=2*\beta$
 $2*\beta = \alpha \wedge$
 $\alpha \leq \beta+10$

2nd iteration

Example: Concolic Testing (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {
```

```
    ← z := double (y);
```

```
    if (z==x) {
```

```
        if (x>y+10) {
```

Crash

```
        } else {    }
```

```
    }
```

```
}
```

Concrete
State

x=30, y=15

Symbolic
State

x= α , y= β

true

3rd iteration

Example: Concolic Testing (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {
```

```
    z := double (y);
```

```
    ← if (z==x) {
```

```
        if (x>y+10) {
```

Crash

```
        } else {    }
```

```
    }
```

```
}
```

Concrete
State

x=30, y=15,
z=30

Symbolic
State

x= α , y= β , z=2* β
true

3rd iteration

Example: Concolic Testing (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {
```

```
    z := double (y);
```

```
    if (z==x) {
```

```
        if (x>y+10) {
```

Crash

```
        } else {    }
```

```
    }
```

```
}
```

Concrete
State

x=30, y=15,
z=30

Symbolic
State

$x=\alpha, y=\beta, z=2*\beta$
 $2*\beta = \alpha$

3rd iteration

Example: Concolic Testing (Cont'd)

```
int double (int v) {  
    return 2*v;  
}
```

```
void testme(int x, int y) {  
    z := double (y);  
    if (z==x) {  
        if (x>y+10) {  
            Crash  
        } else {  
        }  
    }  
}
```

Concrete
State

crashing input

x=30, y=15,
z=30

Symbolic
State

$x=\alpha, y=\beta, z=2*\beta$
 $2*\beta = \alpha \wedge$
 $\alpha > \beta+10$

3rd iteration

Concolic Testing Algorithm

Input : Program P , initial input vector v_0 , budget N

Output: The number of branches covered

```
1:  $T \leftarrow \langle \rangle$ 
2:  $v \leftarrow v_0$ 
3: for  $m = 1$  to  $N$  do
4:    $\Phi_m \leftarrow \text{RunProgram}(P, v)$ 
5:    $T \leftarrow T \cdot \Phi_m$ 
6:   repeat
7:      $(\Phi, \phi_i) \leftarrow \text{Choose}(T)$      $(\Phi = \phi_1 \wedge \dots \wedge \phi_n)$ 
8:   until  $\text{SAT}(\bigwedge_{j < i} \phi_j \wedge \neg \phi_i)$ 
9:    $v \leftarrow \text{model}(\bigwedge_{j < i} \phi_j \wedge \neg \phi_i)$ 
10: end for
11: return  $|\text{Branches}(T)|$ 
```

DSE's Advantage: Unknown/Complex Calls

Recall: “Concrete execution guides symbolic execution to simplify constraints (optimize Φ' before solving it).”

	Concrete State	Symbolic State
<pre>int foo (int v) { return hash(v); }</pre>		
<pre>void testme(int x, int y) { ← x=22, y=7 z := foo (y); if (z==x) { if (x>y+10) { Crash } else { } } }</pre>		<pre>x=α, y=β true</pre>

DSE's Advantage: Unknown/Complex Calls (Cont'd)

Recall: “Concrete execution guides symbolic execution to simplify constraints (optimize Φ' before solving it).”

	Concrete State	Symbolic State
<pre>int foo (int v) { return hash(v); }</pre>		
<pre>void testme(int x, int y) { z := foo (y); ←————— if (z==x) { if (x>y+10) { Crash } else { } } }</pre>	x=22, y=7, z=601...129	x=α, y=β, z=hash(β) true

DSE's Advantage: Unknown/Complex Calls (Cont'd)

Recall: “Concrete execution guides symbolic execution to simplify constraints (optimize Φ' before solving it).”

	Concrete State	Symbolic State
<pre>int foo (int v) { return hash(v); }</pre>		
<pre>void testme(int x, int y) { z := foo (y); if (z==x) { if (x>y+10) { Crash } else { } } } }</pre>	x=22, y=7, z=601...129	x=α, y=β, z=hash(β) hash(β) $\neq$ α

DSE's Advantage: Unknown/Complex Calls (Cont'd)

Recall: "Concrete execution guides symbolic execution to simplify constraints (optimize Φ' before solving it)."

```
int foo (int v) {  
    return hash(v);  
}
```

```
void testme(int x, int y) {  
    z := foo (y);  
  
    if (z==x) {  
        if (x>y+10) {  
            Crash  
        } else {  
            }  
    }  
}
```

Concrete
State

Symbolic
State

- Constraint: $\text{hash}(\beta) = \alpha$
- Replace β by 7: $601\dots129 = \alpha$
- Solution: $\alpha=601\dots129, \beta=7$

$x=22, y=7,$
 $z=601\dots129$

$x=\alpha, y=\beta,$
 $z=\text{hash}(\beta)$
 $\text{hash}(\beta) \neq \alpha$

DSE's Advantage: Unknown/Complex Calls (Cont'd)

Recall: “Concrete execution guides symbolic execution to simplify constraints (optimize Φ' before solving it).”

	Concrete State	Symbolic State
<pre>int foo (int v) { return hash(v); }</pre>		
<pre>void testme(int x, int y) { ← z := foo (y); if (z==x) { if (x>y+10) { Crash } else { } } }</pre>	<pre>x=601...129 y=7</pre>	<pre>x=α, y=β true</pre>

DSE's Advantage: Unknown/Complex Calls (Cont'd)

Recall: “Concrete execution guides symbolic execution to simplify constraints (optimize Φ' before solving it).”

	Concrete State	Symbolic State
<pre>int foo (int v) { return hash(v); }</pre>		
<pre>void testme(int x, int y) { z := foo (y); ←————— if (z==x) { if (x>y+10) { Crash } else { } } }</pre>	<pre>x=601...129 y=7 z=601...129</pre>	<pre>x=α, y=β, z=hash(β) true</pre>

DSE's Advantage: Unknown/Complex Calls (Cont'd)

Recall: “Concrete execution guides symbolic execution to simplify constraints (optimize Φ' before solving it).”

	Concrete State	Symbolic State
<pre>int foo (int v) { return hash(v); }</pre>		
<pre>void testme(int x, int y) { z := foo (y); if (z==x) { ← if (x>y+10) { Crash } else { } } }</pre>	x=601...129 y=7 z=601...129	x=α, y=β, z=hash(β) hash(β) = α

DSE's Advantage: Unknown/Complex Calls (Cont'd)

Recall: "Concrete execution guides symbolic execution to simplify constraints (optimize Φ' before solving it)."

	Concrete State	Symbolic State
<pre>int foo (int v) { return hash(v); }</pre>		
<pre>void testme(int x, int y) { z := foo (y); if (z==x) { if (x>y+10) { Crash } else { } } } }</pre>	<pre>x=601...129 y=7 z=601...129</pre>	<pre>x=α, y=β, z=hash(β) hash(β) = $\alpha \wedge$ $\alpha > \beta + 10$</pre>

DSE's Downside: False Negative

Due to constraints simplified using runtime information, satisfiable constraints might be judged as unsatisfiable during DSE.

	Concrete State	Symbolic State
<pre>int foo (int v) { return secure_hash(v); } void testme(int x, int y) { ← if (x != y) { if (foo(x) == foo(y)) { Crash } } }</pre>	$x=22, y=7$	$x=\alpha, y=\beta$ true

DSE's Downside: False Negative (Cont'd)

Due to constraints simplified using runtime information, satisfiable constraints might be judged as unsatisfiable during DSE.

	Concrete State	Symbolic State
<pre>int foo (int v) { return secure_hash(v); } void testme(int x, int y) { if (x != y) { ← if (foo(x) == foo(y)) { Crash } } }</pre>	$x=22, y=7$	$x=\alpha, y=\beta$ $\alpha \neq \beta$

DSE's Downside: False Negative (Cont'd)

Due to constraints simplified using runtime information, satisfiable constraints might be judged as unsatisfiable during DSE.

	Concrete State	Symbolic State
<pre>int foo (int v) { return secure_hash(v); } void testme(int x, int y) { if (x != y) { if (foo(x) == foo(y)) { Crash } } }</pre>	$x=22, y=7$	$x=\alpha, y=\beta$ $\alpha \neq \beta \wedge$ $\text{hash}(\alpha) \neq \text{hash}(\beta)$

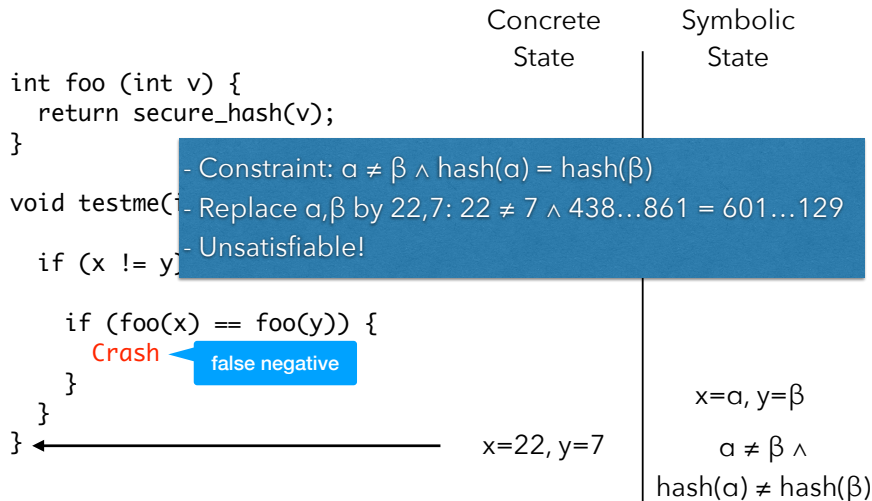
DSE's Downside: False Negative (Cont'd)

Due to constraints simplified using runtime information, satisfiable constraints might be judged as unsatisfiable during DSE.

	Concrete State	Symbolic State
<pre>int foo (int v) { return secure_hash(v); }</pre>		
<pre>void testme(if (x != y)</pre>	- Constraint: $\alpha \neq \beta \wedge \text{hash}(\alpha) = \text{hash}(\beta)$ - Replace α, β by 22,7: $22 \neq 7 \wedge 438\dots861 = 601\dots129$ - Unsatisfiable!	
<pre> if (foo(x) == foo(y)) { Crash } } }</pre>		
<pre> } ←</pre>	$x=22, y=7$	$x=\alpha, y=\beta$ $\alpha \neq \beta \wedge$ $\text{hash}(\alpha) \neq \text{hash}(\beta)$

DSE's Downside: False Negative (Cont'd)

Due to constraints simplified using runtime information, satisfiable constraints might be judged as unsatisfiable during DSE.



Summary

- Symbolic execution is a testing approach based on computational logic.
 - ▶ Key idea: execute programs “symbolically” using “symbolic” inputs
- Main challenges: path explosion, constraint solving
- DSE is an effective hybrid approach that (partly) addresses constraint solving.
 - ▶ Symbolic execution (solving a path constraint) guides concrete execution to increase code coverage.
 - ▶ Concrete execution guides symbolic execution to simplify constraints (optimize a path constraint before solving it).